

כל מספר כאן נמדד היום מול Supabase של הפרודקשן, מהשרת הזה (2 ליבות, 2GB). מה שלא נמדד – כתוב.

\$0.06**אודיו לשיעור דרך OpenRouter**

מדויק מהספק · 14-19 קטעי TTS

85**בקשות DB בשנייה לליבה**

נמדד: CPU 12ms לבקשה

~600**ילדים משוחחים במקביל, שרת web**

הערכה מחושבת, לא נמדדה ישירות

התשובה בקצרה

ה-CPU והזיכרון של השרת כמעט לא משתתפים: כל העבודה היא המתנה ל-Gemini, OpenAI ו-Supabase. אחרי תיקוני היום שרת web יחיד של 512MB מספיק למאות ילדים. מה שקובע את המספר הסופי הוא מכסות הספקים. מכסת ה-TTS של Gemini נגמרה עם worker אחד; דרך OpenRouter אותו מודל רץ בלי מכסה. הצינור הכבד – אודיו, תמונות, וידאו – יצא מהשרת לתור עם worker נפרד. עבודה שנופלת חוזרת לבד.

72% מהפולינג נחסם → backoff

כל 2-8 שניות, מכבד 429. מחכה לבדיקת דפדפן

מכסת TTS Gemini נגמרה → OpenRouter

אותו מודל ואותו קול, בלי מכסה. נבדק ב-3 שיעורים

מצב האימות

עדיין לא

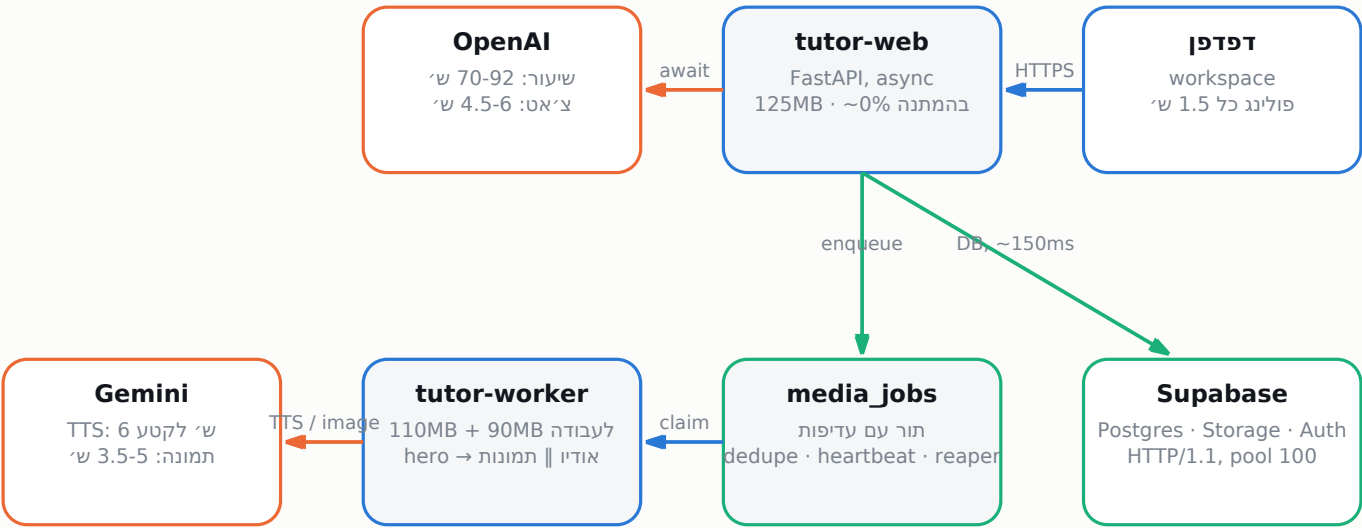
- ☐ מיגרציית ai_calls בפרוד (עלות לכל קריאה)
- ☐ פולינג חדש מהדפדפן האמיתי
- ☐ עם ילדים שונים unit-lesson
- ☐ ראט audio אחרי התיקון (מול הפרוד)
- ☐ משתני סביבה + worker שירות: Render
- ☐ קומיט – רק אחרי כל האמור

אומת מול הפרודקשן

- ☒ תור עבודות: פתיחה → שורה → worker → אודיו מוכן
- ☒ התאוששות מקריסת worker (עבודה יתומה חזרה)
- ☒ ריטריי אוטומטי אחרי כישלון אודיו
- ☒ עומס: 8 ראטי 30, DB במקביל
- ☒ צ'אט: 5 במקביל, 0 שגיאות
- ☒ קריסה יזומה: SIGKILL באמצע, השלמה בניסיון 2
- ☒ שיעורים מלאים + 8 בדיקות 3 OpenRouter: דרך TTS
- ☒ רגרסיה אופליין: 10 חבילות, הכל עובר

"אומת" = רץ מול Supabase של הפרודקשן דרך הראוטים האמיתיים, עם הילד איתן בחשבון שלך.

לפני: השרת החזיר תשובה והמשיך לייצר וידאו ואודיו באותו תהליך. ריסטארט באמצע איבד הכל בשקט. אחרי: הראוט רק מכניס שורה לטבלה. תהליך נפרד מריץ, מדווח, ומתאושש.



מה נכנס לקוד היום

מה	פירוט	מצב
תור media_jobs	5 פונקציות enqueue RPC: עם dedupe, claim, reaper, finish, heartbeat	חדש
worker.py	תהליך נפרד, concurrency לפי SIGTERM env, עדין, מדדים לכל עבודה	חדש
מדדים ב-DB	views, כל 30 ש' service_metrics, לכל בקשה request_log	חדש
event loop	72 קריאות DB סינכרוניות ב-12 ראוטים async → threadpool	תוקן
לקוח Supabase	pool עם HTTP/1.1 → threads יחיד (קריס ב-30 HTTP/2)	תוקן
signed URLs	קריאות לפתיחה → 0-1 batch: לשעה + חתימת אודיו ב cache	תוקן
ייצור כפול	נעילה ל-hero, ראוט audio מכניס לתור במקום לייצר בבקשה	תוקן
ריסריי	ניתוקים בלבד, עבודות: 5 ניסיונות Storage, TTS (כולל 429)	תוקן

הקוד עובד גם בלי המיגרציות: אם הכנסה לתור נכשלת, העבודה רצה בתוך השרת כמו קודם.

ראוט אחרי ראוט. p50 הוא הזמן שהילד מחכה כשעוד 29 מחכים איתו. CPU הוא של תהליך ה-100% web = ליבה.

ראוט	מחכה ל	p50	req/s	CPU	הערה
units (GET)	DB	861ms	27	36%	רשימת יחידות של שיעור
active-lesson-state	DB	512ms	47	28%	איפה הילד עצר
shared-transition	DB	517ms	81	17%	סרטון מעבר משותף
hero-image	DB + cache	1.5s	18	35%	היה 9s ו-30 493MB ייצורים)
visuals	DB + cache	3.2s	11	55%	גדול לכל בקשה. היה 6.7 JSON
unit-lesson	תור + DB	7.4s	5	23%	היה 17s. אותו ילד 30x = תחרות שורות
lesson-intro	תור + DB	2.3s	12	66%	ראוט סינכרוני
audio	תור + DB	—	—	—	תוקן אחרי הריצה: היה מייצר TTS בבקשה
chat	OpenAI	4.5-6s	0.7 × 5	17%	5 במקביל, 0 שגיאות. זמן המודל
tts	Gemini	—	—	—	5/5 נכשלו: מכסה (429)
חדש unit-lesson	OpenAI	70-92s	—	2%	ייצור שיעור. לא תופס thread

מה זה אומר

- ראוט DB עולה כ-CPU 12ms. ליבה אחת: ~85 בקשות בשנייה. שתי ליבות: ~170. הזיכרון לא זז.
- ראוטי המודל לא תופסים thread בזמן ההמתנה. התקרה שלהם היא מכסת OpenAI/Gemini, לא השרת.
- ה-p50 של הראוטים הפשוטים הוא בעיקר סיבובי רשת ל-150ms ~, Supabase, כל אחד, מהשרת הזה בניו-יורק.
- של אותו ילד שמתחרות על שורה אחת progress גדול, וכתיבות JSON: עדיין כבדים i-visuals ו-unit-lesson.

קריאות וכתיבות ל-DB (ספירה מהקוד, לא נמדדה)

פעולה	קריאות	כתיבות	הערה
פתיחת שיעור מה-cache	~7 קריאות	2-3 כתיבות	3 הכנסות לתור (batch 1 · dedupe) חתימה
פולינג audio / visuals	3-4 קריאות	0	אימות + ילד + שיעור, כל 1.5 שניות
הודעת צ'אט	4-5 קריאות	2-3 כתיבות	סשן, היסטוריה, הודעות, usage
שיעור חדש	~7 קריאות	3-4 כתיבות	+ עבודת מדיה: ~40 העלאות ל-Storage

כל קריאה = בקשת HTTPS ל-Supabase. הכפלה ב-150ms נותנת את זמן הרשת המינימלי של הפעולה.

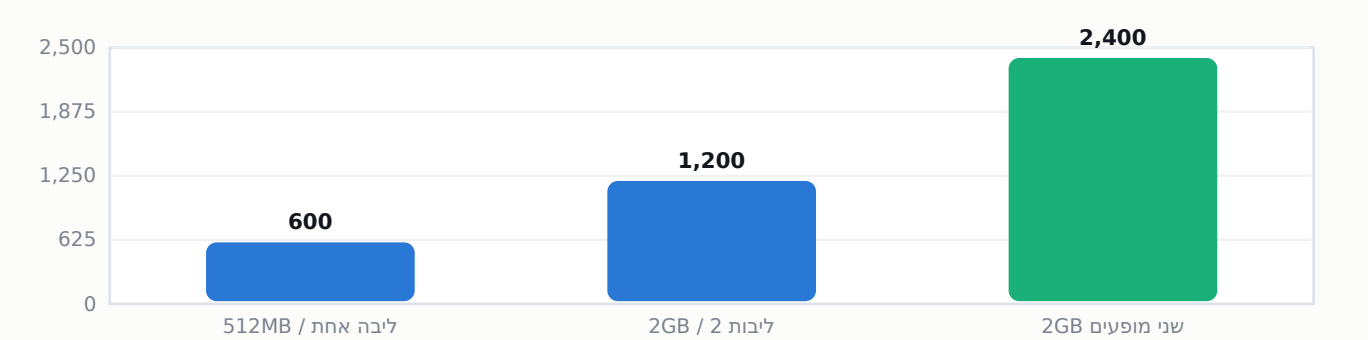
מה עולה ילד אחד

מה	קצב	עלות	הערה
שיחה שגרתית	~3 בקשות בדקה	הודעה כל 30-60 ש' + TTS לתשובה	זניח: ~0.6ms CPU בדקה
פתיחת שיעור חדש	עד 160 בקשות בדקה	פולינג audio + visuals + תמונה-תמונה	פי 50. נמדד: 72% נחסמים
שיעור חדש (מדיה)	פעם אחת לשיעור, לא לילד	~130 ש' worker, 200MB שיא	משותף לכל הכיתה
סרטוני פתיחה	פעם אחת לילד	3 סרטונים, ~139 ש'	עדיפות נמוכה, יש fallback

זיכרון ו-CPU לתהליך (נמדד)

תהליך	זיכרון	CPU	הערה
tutor-web	בעומס 105MB → 152MB	0-2% בהמתנה, 36-66% ב-30 במקביל	מקסימום שנמדד: 152MB
tutor-worker	לעבודה ~90MB + 103MB	3-5% ממוצע, 30% שיא	שיא 211MB בעבודה אחת

מה יוצא מזה: ילדים משוחחים במקביל, לפי תצורת שרת web



חישוב שמרני: 30 בקשות בשנייה לליבה ÷ 3 בקשות לדקה לילד. מעל זה הגבול הוא מכסת OpenAI, לא השרת.

worker	שיעורים חדשים לשעה	הערה
512MB, concurrency 3	~85	מספיק ליום שגרתי
2GB, concurrency 12	~330	גל של כיתה חדשה
3 מכונות × 2GB	~1,000	מכאן מגבילה מכסת Gemini
מכסת Gemini TTS היום	נגמרה	עם worker אחד. חובה להעלות

130 שניות לשיעור ÷ concurrency. לפי שיעורים חדשים, לא לפי ילדים: שיעור מוכן משרת את כל הכיתה.

שורה תחתונה

- השרת אינו הגבול. אחרי היום, מופע web אחד מחזיק מאות ילדים; שניים נדרשים לשרידות, לא לקיבולת.
- הגבול הוא מכסות: Gemini TTS נגמרה עם worker אחד. OpenAI טרם נבדק בעומס (5 במקביל עברו).
- הפולינג בפרונט הוא הבזבז הגדול: פי 50 מבקשות השיחה, ורובו נחסם. תיקון קטן, השפעה גדולה.

עלויות: כמה עולה כל קריאה

נמדד היום, ומה נרשם מעכשיו לכל קריאה

מהיום כל קריאת מודל נרשמת בטבלת ai_calls: ספק, מודל, מטרה, ילד, שיעור, טוקנים, שניות אודיו, latency ועלות. דרך OpenRouter העלות מדויקת מהספק. ישירות מול OpenAI ו-Gemini היא הערכה מטבלת מחירים. בלי מחיר – מסומן.

\$10

קרדיט בחשבון OpenRouter

ללא תקרת בקשות

\$0.27

היום OpenRouter

כ-80 קריאות 3, TTS שיעורים + בדיקות

\$0.0033

קטע TTS של 6 שניות

מדויק: 165 טוקני פלט OpenRouter,

מה עולה שיעור חדש אחד (14-19 קטעי אודיו, 14-21 תמונות)

מה	קריאות	עלות	מקור
אודיו, OpenRouter	14-19 קריאות	\$0.05-0.06	נמדד
אודיו, Gemini ישיר	אותן קריאות	~\$0.03-0.04 לפי \$10/M	הערכה
ייצור השיעור, gpt-5.6-sol	2-6 קריאות	לפי \$5/\$30 למיליון; נכנס ל-usage_summary	הערכה
תמונות, flash-lite-image	14-21 + hero	אין מחיר בקוד — AI_IMAGE_PRICES_JSON	לא ידוע
סרטוני פתיחה, לילד	3 סרטונים	אין מחיר בקוד	לא ידוע
צ'אט, gpt-4o-mini	הודעה	~\$0.0003-0.001 להודעה לפי \$0.15/\$0.60	הערכה

"נמדד" = מהספק לכל קריאה. "הערכה" = טבלת מחירים בקוד (MODEL_PRICING_USD). "לא ידוע" = חסר מחיר; ai_costs_daily סופר.

מה נרשם מעכשיו, ואיפה לראות

view /	מה יש שם
ai_calls	שורה לכל קריאה: provider, model, purpose, kid, lesson, tokens, seconds, cost
ai_costs_daily	לפי יום × ספק × מודל × מטרה: קריאות, שגיאות, טוקנים, latency ממוצע, עלות
ai_costs_per_lesson	כמה עלה כל שיעור: מדיה לחוד, סה"כ, מתי
ai_costs_per_kid	כמה עלה כל ילד ליום
usage_summary	הסיכום הישן נשאר: OpenAI/Gemini למשתמש, עלות לסשן ב-tutor_sessions

מה זה אומר לאלף ילדים

- המדיה היא לשיעור, לא לילד. כיתה שלמה של 150 יחידות: פעם אחת ~\$9 אודיו + תמונות, ואז אפס.
- העלות המשתנה לילד היא הצ'אט: ~30 הודעות ביום × ~\$0.0005 ≈ 1.5 סנט ליום, ~\$0.45 לחודש.
- סרטון פתיחה אישי: פעם אחת לילד, 3 סרטונים. המחיר לא בקוד — למלא כדי שייספר.
- להשלמת התמונה: להריץ את מיגרציית ai_calls, ואחרי יום אחד ai_costs_daily נותן את הכל במקום הערכות.

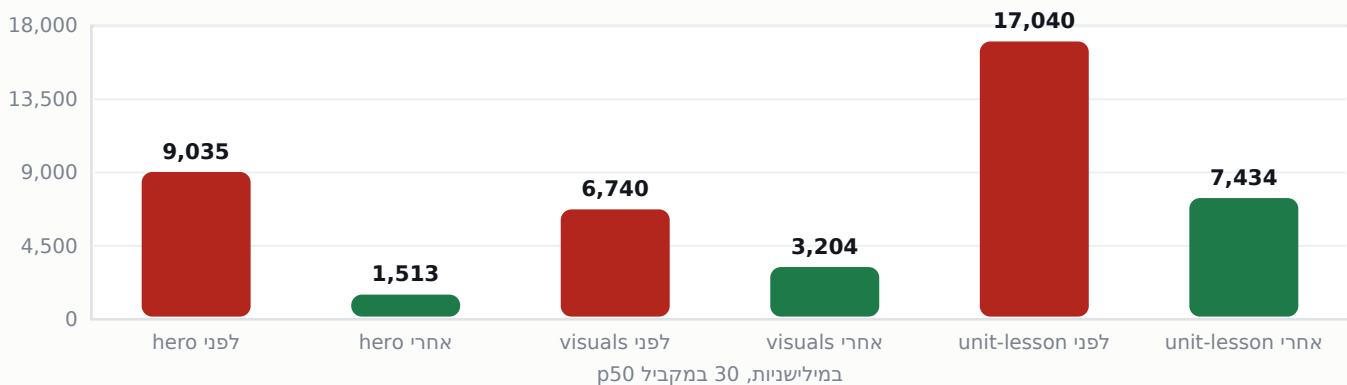
מה נמצא ומה תוקן

לפי סדר הגילוי, עם המדידה שחשפה כל אחד

ממצא	מה קרה	תיקון	מצב
עבודות רקע בתוך השרת	ריסטארט = מדיה אבודה	תור + worker + reaper	אומת
worker אחד, FIFO	אודיו חיכה 2:20 מאחורי וידאו	עדיפויות: אודיו 10, וידאו 60	אומת SQL
מזדמן TTS 400	כישלון אחד = כל האודיו נופל	ריטריי 4 ניסיונות	אומת
מכסה TTS 429	נגמר אחרי 30 ייצורים במקביל	אותו מודל, בלי מכסה, OpenRouter	אומת
"Server disconnected" Storage	HTTP/2 שנזנח אחרי דקה	ריטריי + HTTP/1.1 pool	אומת
כישלון אודיו נבלע	עבודה "done", שיעור בלי אודיו	העבודה נכשלת ורצה שוב	אומת
event loop חסום	~3 בקשות בשנייה לכל התהליך	72 קריאות ל-threadpool	אומת
לפתיחה 40 signed URLs	30 פתיחות = 18 שניות	cache + batch	אומת
ייצורים hero: 30	ש' 20, 121% CPU, 493MB	נעילה לשיעור	אומת
בבקשה audio: TTS 30x	27 × 500, 3 דקות, מכסה	תור במקום ייצור בבקשה	אופליין
עבודה בלי אודיו = done	שיעור אילם בלי retry	אודיו לא מוכן = כישלון, retry	אופליין
לנצח "generating"	מת = שיעור תקוע worker	מעל 10 דק' נלקח מחדש	אופליין
פולינג < מגביל קצב	72% מהבקשות 429	ש', 20 ש' על 429 2-8 backoff	בקוד
גיל מול כיתה	ילד עם age=7 חסום מכל שיעור	לבדוק את המודל	פתוח

"אומת" = נבדק מול הפרודקשן. "אופליין" = נבדק עם Supabase מדומה, מחכה למכסת TTS לבדיקה אמיתית.

שלוש הריצות שהיו לפני/אחרי



לפני שממשיכים

- 1 להריץ את מיגרציית ai_calls (יש rollback), ולמלא מחירים לתמונות ולווידאו. אז העלות מלאה.
- 2 להשלים את האימות: פולינג ו-audio מהדפדפן, ילדים שונים. ואז קומיט.
- 3 אותם משתני סביבה, python worker.py, פקודה Background Worker שירות: Render.

שבוע ראשון

- 4 אין מכסה שמונעת OpenRouter קטעים בטור = 100-180 ש'; 4 במקביל $\approx$ 40 ש'. עם 14-19 worker-במקביל ב TTS.
- 5 אודיו הדרגתי: לכתוב לפי חלקים ולהתחיל כשחלק 1 מוכן. חוצה את ההמתנה מ-105 ל-50 שניות.
- 6 שרידות, לא קיבולת. Redis-מגביל קצב ב web, שני מופעי, concurrency עם worker Render: 2+.
- 7 כבר יש 90 שניות TTS Gemini ל-OpenAI. מפורש על כל קריאת timeout.

כשמגיעים לאלפים

- 8 מכסות OpenAI: לבדוק RPM/TPM של המודלים בשימוש ולבקש העלאה מראש. backoff על 429 בתור.
- 9 ב-11 בקשות בשנייה CPU שחוזר. היום זה 55%-JSON לצמצם את ה visuals ו-unit-lesson.
- 10 ניטור: request_log_hourly ו-service_metrics_5min כבר בטבלאות. לחבר לוח מחוונים.
- 11 הצינור והמודל של התהליך לא מתאימים לו: Lambda לא serverless. אם רוצים Cloud Run.

כלים שנשארים

כלי	מה הוא נותן
tools/e2e_media_jobs.py	מקצה לקצה: פתיחה, תור, worker, אודיו. --load, --kill-worker-once, fresh,
tools/load_api.py	עומס לכל ראוט, סימולציית ילדים (--children), ראוטי (--with-ai) (AI)
tools/async_db_helpers.py	מעביר קריאות DB סינכרוניות בראוטים async ל-threadpool. להריץ אחרי ראוט חדש
/var/log/iakids/	לוגים של כל ריצה: CPU/RSS, worker, server, כל 2 ש'), phases,
request_log · service_metrics	בטבלאות Supabase, לכל בקשה ולכל תהליך. ניקוי אחרי 30 יום

המקור לכל המספרים: handoff_performance.md בשורש הריפו, והלוגים תחת /var/log/iakids/.